

Mini Video Processing Backend using AWS and Terraform

1. Introduction

With the rapid growth of video-based applications, efficient storage, processing, and delivery of video content have become essential requirements for modern web platforms. Traditional approaches often rely on dedicated servers for handling uploads and video processing tasks, which can be costly, difficult to scale, and require continuous maintenance.

This project, titled "**Mini Video Processing Backend using AWS Serverless Architecture and Terraform**," was developed to demonstrate how cloud-native services can be combined to build a scalable, reliable, and cost-effective video processing system. The application enables users to upload videos securely, automatically generate thumbnails, store metadata, and retrieve processed content through a web interface.

The system leverages several AWS services, including Amazon S3 for object storage, AWS Lambda for serverless computing, Amazon SQS for asynchronous message processing, Amazon DynamoDB for metadata storage, and Amazon CloudWatch for monitoring and alerting. Infrastructure provisioning and management are automated using Terraform, following Infrastructure as Code (IaC) principles.

The architecture is designed around an event-driven workflow. When a user uploads a video, the upload triggers an automated processing pipeline that generates a thumbnail and updates the corresponding metadata without requiring manual intervention. This approach improves scalability, reduces operational overhead, and ensures efficient resource utilization.

The primary objective of this project is to gain practical experience with serverless application development, cloud architecture design, event-driven systems, and Infrastructure as Code while building a real-world backend application that follows industry best practices.

2. Problem Statement

Modern applications increasingly rely on video content for communication, education, entertainment, and social interaction. However, managing video uploads and processing presents several challenges. Video files are typically large in size, require significant storage capacity, and often need additional processing such as thumbnail generation before they can be displayed effectively to users.

Traditional video processing systems usually depend on dedicated servers that continuously run in the background to handle uploads, process media files, and manage metadata. Such systems

can be expensive to maintain, difficult to scale during periods of high demand, and prone to resource wastage when traffic is low.

Another challenge is ensuring that video processing tasks do not negatively impact the user experience. If thumbnail generation or metadata updates are performed synchronously during uploads, users may experience increased response times and delays. Additionally, failures during processing can result in inconsistent data and poor system reliability.

The objective of this project is to design and implement a scalable, event-driven, and serverless video processing backend that addresses these challenges. The system should:

- Enable secure and efficient video uploads.
- Automatically generate thumbnails for uploaded videos.
- Store and manage video metadata effectively.
- Process videos asynchronously without affecting user experience.
- Handle failures reliably using queue-based architecture.
- Scale automatically based on workload.
- Minimize infrastructure management and operational costs.

To achieve these goals, the project utilizes AWS serverless services and Infrastructure as Code (IaC) principles through Terraform, creating a modern cloud-native solution for video processing.

3. System Architecture

3. System Architecture

3.1 Overview

The Mini Video Processing Backend follows a serverless, event-driven architecture built entirely on AWS cloud services. The system is designed to securely handle video uploads, process videos asynchronously, generate thumbnails automatically, store metadata, and provide users with access to uploaded content.

The architecture separates user-facing operations from resource-intensive processing tasks using Amazon SQS, ensuring scalability, reliability, and improved user experience.

The system consists of the following major components:

- Frontend Application (React)
- AWS Lambda Functions
- Amazon S3 Buckets
- Amazon SQS Queues

- Amazon DynamoDB
 - Amazon CloudWatch
 - Terraform Infrastructure
-

3.2 Architectural Components

3.2.1 Frontend Application

The frontend provides the user interface for interacting with the system.

Functions:

- Display uploaded videos
- Display video thumbnails
- Upload videos
- View videos
- Show processing status

The frontend never directly accesses S3 buckets. Instead, it communicates with backend APIs exposed through Lambda functions.

3.2.2 Generate Upload URL Lambda

Function Name:

generate-upload-url

Purpose:

This Lambda generates a pre-signed URL that allows users to upload videos directly to Amazon S3.

Workflow:

1. User clicks Upload.
2. Frontend calls Generate Upload URL Lambda.
3. Lambda generates a secure S3 pre-signed URL.
4. URL is returned to the frontend.
5. Frontend uploads the video directly to S3.

Benefits:

- No video data passes through application servers.

- Reduced backend load.
 - Improved upload performance.
 - Secure temporary access.
-

3.2.3 Upload Bucket (Amazon S3)

Bucket:

akshay-video-upload-bucket-2026

Purpose:

Stores uploaded video files.

Functions:

- Durable video storage
- Event generation
- Source for thumbnail generation

Object Structure:

videos/

Example:

videos/sample-video.mp4

Events:

Object creation events trigger S3 notifications.

3.2.4 S3 Event Notification

Purpose:

Detects when a new video is uploaded.

Configuration:

Event Type:

s3:ObjectCreated:*

Filter Prefix:

videos/

Filter Suffix:

.mp4

Action:

Send message to SQS queue.

Benefits:

- Decouples upload and processing stages.
- Enables asynchronous processing.

3.2.5 Video Processing Queue (Amazon SQS)

Queue:

video-processing-queue

Purpose:

Acts as a buffer between uploads and processing.

Functions:

- Stores upload events
- Decouples services
- Handles traffic spikes
- Provides retry capability

Workflow:

1. S3 sends message to queue.
2. Queue stores upload event.
3. Lambda retrieves messages for processing.

Advantages:

- Reliable message delivery
 - Fault tolerance
 - Automatic scaling
-

3.2.6 Event Source Mapping

Purpose:

Connects the SQS queue with the Thumbnail Generator Lambda.

Configuration:

Batch Size:

1

Function:

video-thumbnail-generator-v2

Responsibilities:

- Poll SQS queue
 - Invoke Lambda automatically
 - Manage retries
 - Handle failed messages
-

3.2.7 Thumbnail Generator Lambda

Function Name:

video-thumbnail-generator-v2

Purpose:

Core processing engine of the application.

Responsibilities:

- Read SQS messages
- Download video from S3
- Generate thumbnail

- Upload thumbnail to S3
- Update DynamoDB metadata

Runtime:

Node.js

Layer:

Custom FFmpeg Layer

Environment Variables:

- DYNAMODB_TABLE
- THUMBNAIL_BUCKET

Processing Flow:

1. Receive SQS message.
2. Extract uploaded video key.
3. Download video from Upload Bucket.
4. Store video temporarily in Lambda /tmp directory.
5. Execute FFmpeg.
6. Generate thumbnail image.
7. Upload thumbnail to Thumbnail Bucket.
8. Update video metadata in DynamoDB.
9. Mark status as completed.

Benefits:

- Fully serverless processing.
- Automatic scaling.
- Cost efficient.

3.2.8 FFmpeg Layer

Purpose:

Provides video processing capabilities.

Functions:

- Extract video frames
- Generate thumbnails

Reason for Layer:

AWS Lambda does not include FFmpeg by default.

Custom Layer:

custom-ffmpeg-layer

3.2.9 Thumbnail Bucket (Amazon S3)

Bucket:

akshay-video-thumbnails-2026

Purpose:

Stores generated thumbnails.

Functions:

- Persistent image storage
- Thumbnail retrieval

Output Example:

thumbnails/video1.jpg

3.2.10 DynamoDB Table

Table:

videos

Purpose:

Stores metadata related to uploaded videos.

Primary Key:

video_id

Stored Attributes:

- video_id
- video_name
- upload_time
- thumbnail_url
- status

Example Status Values:

- Processing
- Completed
- Failed

Benefits:

- Fast lookups
- Serverless scaling
- Low latency

3.2.11 List Videos Lambda

Function Name:

list-videos

Purpose:

Retrieves video metadata from DynamoDB.

Responsibilities:

- Scan DynamoDB table
- Return processed video records
- Provide thumbnail information

Used By:

Frontend application

3.2.12 Generate View URL Lambda

Function Name:

generate-view-url

Purpose:

Generates secure pre-signed URLs for viewing videos.

Workflow:

1. User selects a video.
2. Frontend requests viewing URL.
3. Lambda generates temporary S3 access URL.
4. URL is returned to frontend.
5. Video is streamed securely.

Benefits:

- Videos remain private.
 - Controlled access.
 - Improved security.
-

3.2.13 Failure Handler Lambda

Function Name:

video-failure-handler

Purpose:

Processes failed video processing events.

Responsibilities:

- Handle failed messages
- Log failures
- Update failure status

Benefits:

- Improved reliability
 - Easier troubleshooting
-

3.2.14 Dead Letter Queue (DLQ)

Queue:

video-processing-dlq

Purpose:

Stores messages that cannot be processed successfully.

Functions:

- Failure isolation
- Debugging support
- Message retention

Advantages:

- Prevents message loss.
 - Improves fault tolerance.
-

3.2.15 CloudWatch Logs

Purpose:

Centralized logging system.

Captures:

- Lambda execution logs
- Error messages
- Processing details
- Debug information

Benefits:

- Troubleshooting
 - Performance monitoring
 - Operational visibility
-

3.2.16 CloudWatch Alarm

Alarm:

DLQ-Messages-Alarm

Purpose:

Monitors the Dead Letter Queue.

Condition:

Messages in DLQ $\geq$ 1

Action:

Alert administrators of processing failures.

Benefits:

- Early failure detection
 - Improved reliability
-

3.2.17 Terraform

Purpose:

Infrastructure as Code (IaC).

Managed Resources:

- S3 Buckets
- DynamoDB Table
- SQS Queues
- Lambda Functions
- Event Source Mapping
- S3 Notifications
- CloudWatch Alarm

Benefits:

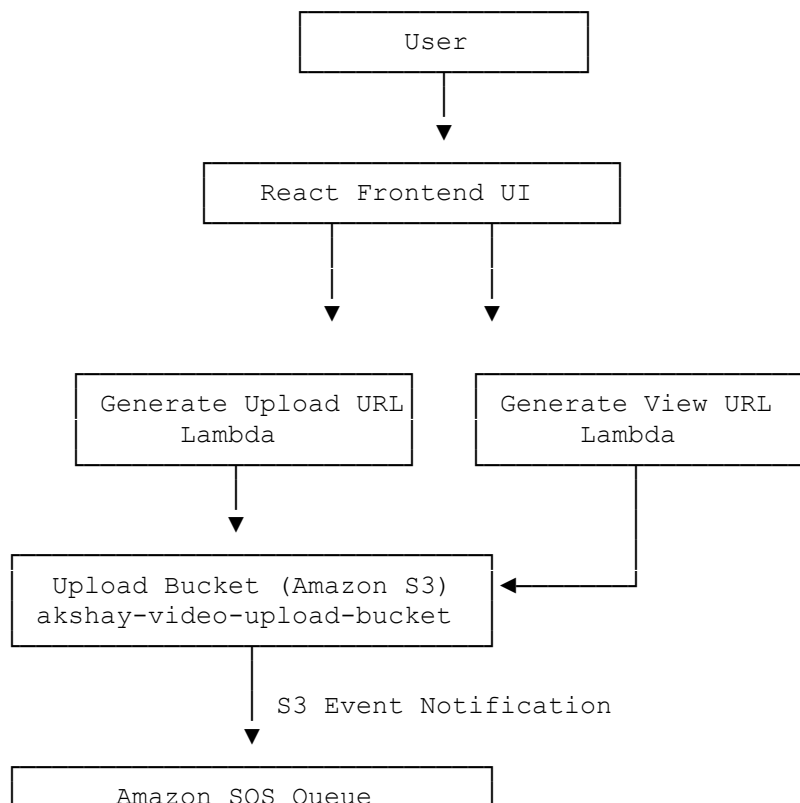
- Reproducible infrastructure
- Version control
- Faster deployments

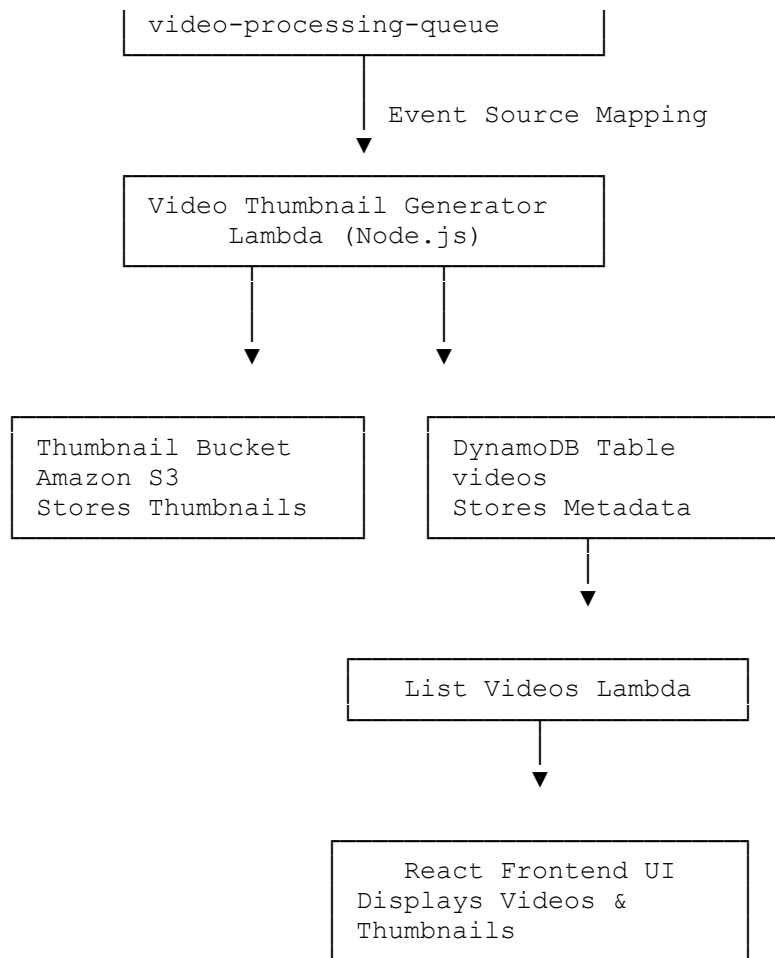
- Reduced manual configuration

3.3 End-to-End Workflow

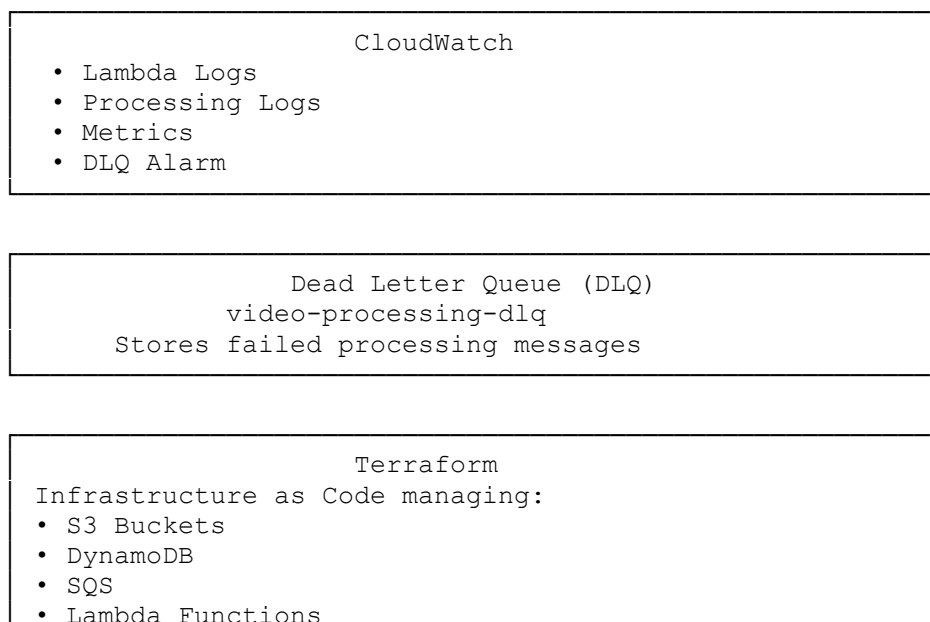
1. User uploads a video from the frontend.
2. Generate Upload URL Lambda creates a pre-signed URL.
3. Video is uploaded to S3 Upload Bucket.
4. S3 triggers an event notification.
5. Event is sent to SQS Queue.
6. Event Source Mapping invokes Thumbnail Generator Lambda.
7. Lambda downloads the video.
8. FFmpeg generates a thumbnail.
9. Thumbnail is uploaded to Thumbnail Bucket.
10. DynamoDB metadata is updated.
11. User requests video list.
12. List Videos Lambda retrieves metadata.
13. Frontend displays videos and thumbnails.
14. User selects a video.
15. Generate View URL Lambda creates a secure viewing URL.
16. Frontend streams the video securely from S3.

3.4 Architecture Diagram





Monitoring & Reliability



- Event Source Mapping
- S3 Notifications
- CloudWatch Alarm

Figure 3.1: Overall architecture of the Mini Video Processing Backend built using AWS Serverless Services and Terraform.

4. Technologies Used

This project utilizes a combination of cloud services, programming languages, frameworks, and infrastructure automation tools to build a scalable and serverless video processing system.

4.1 Amazon Web Services (AWS)

Amazon Web Services (AWS) serves as the cloud platform for the entire project. AWS provides scalable, reliable, and fully managed services that eliminate the need to manage physical servers.

The project uses multiple AWS services including Amazon S3, AWS Lambda, Amazon SQS, Amazon DynamoDB, and Amazon CloudWatch to implement a fully serverless architecture.

4.2 Amazon S3 (Simple Storage Service)

Amazon S3 is used for object storage.

Purpose

- Store uploaded video files
- Store generated thumbnail images
- Provide durable and highly available storage

Buckets Used

1. akshay-video-upload-bucket-2026
 - Stores uploaded videos
2. akshay-video-thumbnails-2026
 - Stores generated thumbnails

Benefits

- High durability

- Unlimited scalability
 - Event notification support
 - Cost-effective storage
-

4.3 AWS Lambda

AWS Lambda is the serverless compute service used to execute backend logic without managing servers.

Lambda Functions Used

1. generate-upload-url
2. generate-view-url
3. list-videos
4. thumbnail-creation
5. video-thumbnail-generator-v2
6. video-failure-handler

Benefits

- Serverless execution
 - Automatic scaling
 - Pay-per-use pricing
 - Reduced operational overhead
-

4.4 Amazon SQS (Simple Queue Service)

Amazon SQS is used for asynchronous communication between services.

Queues Used

1. video-processing-queue
2. video-processing-dlq

Purpose

- Decouple upload and processing stages
- Buffer incoming events
- Improve reliability
- Handle failures gracefully

Benefits

- Fault tolerance
 - Reliable message delivery
 - Automatic scaling
-

4.5 Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database service.

Table Used

videos

Purpose

- Store video metadata
- Store processing status
- Store thumbnail information

Benefits

- Low latency
 - Automatic scaling
 - Serverless database management
-

4.6 Amazon CloudWatch

Amazon CloudWatch provides monitoring and observability capabilities.

Features Used

- Lambda logs
- Application logs
- Metrics monitoring
- CloudWatch alarms

Configured Alarm

DLQ-Messages-Alarm

Benefits

- Centralized monitoring
 - Failure detection
 - Operational visibility
-

4.7 Terraform

Terraform is used as the Infrastructure as Code (IaC) tool.

Managed Resources

- S3 Buckets
- Lambda Functions
- DynamoDB Table
- SQS Queues
- Event Source Mapping
- S3 Notifications
- CloudWatch Alarm

Benefits

- Infrastructure automation
 - Version-controlled deployments
 - Reproducible environments
 - Reduced manual configuration
-

4.8 React.js

React.js is used for building the frontend user interface.

Features Implemented

- Video upload interface
- Video listing page
- Thumbnail display
- Video viewing functionality

Benefits

- Component-based architecture

- Fast rendering
 - Improved user experience
-

4.9 Python

Python is used for several Lambda functions.

Usage

- Generate Upload URL
- Generate View URL
- List Videos
- Thumbnail Metadata Processing

Benefits

- Simple syntax
 - AWS SDK support
 - Fast development
-

4.10 Node.js

Node.js is used for video processing functions.

Usage

- Video Thumbnail Generator
- Failure Handler

Benefits

- Efficient asynchronous processing
 - Strong AWS SDK integration
 - Suitable for event-driven workloads
-

4.11 FFmpeg

FFmpeg is an open-source multimedia processing framework.

Purpose

- Extract frames from videos
- Generate thumbnail images

Implementation

- Deployed as a custom AWS Lambda Layer

Benefits

- High-performance video processing
 - Industry-standard multimedia tool
-

4.12 Git and GitHub

Git is used for version control, while GitHub is used as the remote repository.

Purpose

- Source code management
- Version tracking
- Collaboration
- Backup and recovery

Benefits

- Change tracking
 - Branch management
 - Secure code storage
-

4.13 Visual Studio Code

Visual Studio Code (VS Code) is the primary development environment used during the project.

Usage

- Backend development
- Frontend development
- Terraform configuration
- AWS infrastructure management

Benefits

- Lightweight IDE
- Extensive extension support
- Integrated terminal
- Git integration

4.14 AWS SDK

The AWS Software Development Kit (SDK) is used by Lambda functions to interact with AWS services.

Usage

- S3 operations
- DynamoDB operations
- SQS processing
- Presigned URL generation

Benefits

- Simplified AWS service integration
- Secure API access
- Improved developer productivity

5. AWS Resources Used

The Mini Video Processing Backend is built entirely on AWS cloud services following a serverless and event-driven architecture. Each AWS resource serves a specific purpose within the video processing workflow.

5.1 Amazon S3 Buckets

Amazon S3 is used as the storage layer of the application.

5.1.1 Video Upload Bucket

Resource Name:

akshay-video-upload-bucket-2026

Purpose:

This bucket stores all uploaded video files submitted by users through the frontend application.

Responsibilities:

- Store video objects
- Trigger processing events
- Act as the source for thumbnail generation

Example Object Path:

videos/sample-video.mp4

5.1.2 Thumbnail Bucket

Resource Name:

akshay-video-thumbnails-2026

Purpose:

This bucket stores thumbnail images generated from uploaded videos.

Responsibilities:

- Store generated thumbnails
- Provide thumbnail access to frontend users
- Maintain processed media assets

Example Object Path:

thumbnails/sample-video.jpg

5.2 AWS Lambda Functions

AWS Lambda provides serverless compute capabilities for the application.

5.2.1 Generate Upload URL Lambda

Function Name:

generate-upload-url

Runtime:

Python 3.13

Purpose:

Generates pre-signed upload URLs that allow users to upload videos directly to Amazon S3.

Responsibilities:

- Receive upload requests
 - Generate secure upload URLs
 - Return URLs to frontend
-

5.2.2 Generate View URL Lambda

Function Name:

generate-view-url

Runtime:

Python 3.13

Purpose:

Generates temporary viewing URLs for stored videos.

Responsibilities:

- Receive video viewing requests
 - Generate pre-signed S3 URLs
 - Return URLs securely
-

5.2.3 List Videos Lambda

Function Name:

list-videos

Runtime:

Python 3.13

Purpose:

Retrieves video metadata from DynamoDB.

Responsibilities:

- Query DynamoDB
 - Return video details
 - Provide thumbnail information
-

5.2.4 Thumbnail Creation Lambda

Function Name:

thumbnail-creation

Runtime:

Python 3.13

Purpose:

Processes thumbnail-related metadata operations and database updates.

Responsibilities:

- Manage processing information
 - Update metadata records
-

5.2.5 Video Thumbnail Generator Lambda

Function Name:

video-thumbnail-generator-v2

Runtime:

Node.js 24.x

Purpose:

Acts as the primary video processing engine.

Responsibilities:

- Receive SQS messages
- Download uploaded videos
- Generate thumbnails using FFmpeg
- Upload thumbnails to S3
- Update DynamoDB records

Configuration:

- Memory: 1024 MB
 - Timeout: 60 seconds
 - Custom FFmpeg Layer Attached
-

5.2.6 Failure Handler Lambda

Function Name:

video-failure-handler

Runtime:

Node.js 24.x

Purpose:

Handles processing failures and error scenarios.

Responsibilities:

- Process failed events
- Log failures
- Support troubleshooting

5.3 Amazon SQS Queues

Amazon SQS provides asynchronous communication between services.

5.3.1 Video Processing Queue

Queue Name:

video-processing-queue

Purpose:

Acts as the main processing queue.

Responsibilities:

- Receive S3 upload events
- Store processing requests
- Trigger thumbnail generation

Benefits:

- Decouples services
 - Supports automatic retries
 - Improves reliability
-

5.3.2 Dead Letter Queue (DLQ)

Queue Name:

video-processing-dlq

Purpose:

Stores messages that fail processing after multiple retry attempts.

Responsibilities:

- Preserve failed messages
- Prevent message loss

- Assist debugging
-

5.4 Amazon DynamoDB

Table Name:

videos

Purpose:

Stores metadata related to uploaded videos.

Partition Key:

video_id

Stored Information:

- Video ID
- Video Name
- Upload Timestamp
- Processing Status
- Thumbnail URL

Benefits:

- Low-latency access
 - Automatic scaling
 - Serverless operation
-

5.5 Amazon CloudWatch

Amazon CloudWatch provides monitoring, logging, and alerting capabilities.

CloudWatch Logs

Used for:

- Lambda execution logs
- Error tracking
- Processing diagnostics

CloudWatch Metrics

Used for:

- Queue monitoring
 - Lambda performance tracking
 - System health monitoring
-

CloudWatch Alarm

Alarm Name:

DLQ-Messages-Alarm

Purpose:

Monitors the Dead Letter Queue for failed processing events.

Trigger Condition:

Number of messages in DLQ $\geq$ 1

Benefits:

- Early failure detection
 - Faster incident response
-

5.6 Lambda Event Source Mapping

Purpose:

Connects the SQS queue directly to the Thumbnail Generator Lambda.

Configuration:

- Source Queue: video-processing-queue
- Target Lambda: video-thumbnail-generator-v2
- Batch Size: 1

Responsibilities:

- Poll messages from SQS

- Invoke Lambda automatically
- Manage retries

5.7 S3 Event Notification

Purpose:

Automatically sends events to Amazon SQS whenever a new video is uploaded.

Configuration:

Event Type:

s3:ObjectCreated:*

Filter Prefix:

videos/

Filter Suffix:

.mp4

Destination:

video-processing-queue

Benefits:

- Fully automated processing pipeline
- Event-driven architecture
- No manual intervention required

5.8 AWS Lambda Layer

Layer Name:

custom-ffmpeg-layer

Purpose:

Provides FFmpeg binaries required for video processing.

Responsibilities:

- Frame extraction
- Thumbnail generation

Benefits:

- Reusable across functions
- Reduces deployment package size

5.9 Infrastructure Management

Infrastructure as Code Tool:

Terraform

Terraform manages the following AWS resources:

- S3 Upload Bucket
- S3 Thumbnail Bucket
- DynamoDB Table
- SQS Queue
- Dead Letter Queue
- Lambda Functions
- Event Source Mapping
- S3 Event Notifications
- CloudWatch Alarm

This ensures consistent, reproducible, and version-controlled infrastructure deployment.

6. Step-by-Step Workflow

6.1 Overview

The Mini Video Processing Backend follows an event-driven workflow where multiple AWS services work together to process uploaded videos automatically. The entire process is serverless and requires no manual intervention after a user uploads a video.

The workflow begins when a user uploads a video through the frontend application and ends when the processed video information and thumbnail are displayed to the user.

6.2 Video Upload Workflow

Step 1: User Initiates Upload

The user accesses the frontend application and selects a video file to upload.

Input:

- Video file (.mp4)

Actor:

- User
- React Frontend

Output:

- Upload request generated
-

Step 2: Frontend Requests Upload URL

The frontend sends a request to the Generate Upload URL Lambda.

Purpose:

- Obtain a secure temporary upload URL

AWS Resource:

- generate-upload-url Lambda

Output:

- Pre-signed S3 Upload URL
-

Step 3: Upload URL Generation

The Generate Upload URL Lambda creates a temporary pre-signed URL using the AWS SDK.

Responsibilities:

- Generate unique video ID

- Create S3 object path
- Generate secure upload URL

AWS Services Used:

- Lambda
- S3

Output:

- Upload URL returned to frontend
-

Step 4: Video Upload to S3

The frontend uploads the selected video directly to the Upload Bucket using the pre-signed URL.

AWS Resource:

- akshay-video-upload-bucket-2026

Example Path:

videos/sample-video.mp4

Benefits:

- Faster uploads
- Reduced backend load
- Improved scalability

Output:

- Video successfully stored in S3
-

6.3 Event Processing Workflow

Step 5: S3 Event Triggered

After successful upload, Amazon S3 automatically generates an Object Created event.

Event Type:

s3:ObjectCreated:*

Trigger Condition:

- New .mp4 file uploaded
- File located under videos/ folder

Output:

- Event notification created
-

Step 6: Event Sent to SQS

S3 forwards the upload event to the Video Processing Queue.

AWS Resource:

video-processing-queue

Purpose:

- Decouple upload and processing
- Prevent system overload
- Enable retries

Output:

- Message stored in queue
-

Step 7: Event Source Mapping Polls Queue

Lambda Event Source Mapping continuously monitors the SQS queue.

Configuration:

- Batch Size = 1

Responsibilities:

- Poll messages
- Invoke processing Lambda

Output:

- Lambda invocation request

Step 8: Thumbnail Generator Lambda Invoked

The Video Thumbnail Generator Lambda is triggered automatically.

AWS Resource:

video-thumbnail-generator-v2

Purpose:

- Process uploaded videos
- Generate thumbnails

Input:

- SQS message containing video details

Output:

- Processing begins

6.4 Video Processing Workflow

Step 9: Video Download

The Thumbnail Generator Lambda retrieves the uploaded video from the Upload Bucket.

AWS Services:

- Lambda
- S3

Actions:

- Read S3 object key
- Download video file
- Store temporarily in /tmp directory

Output:

- Local video copy available

Step 10: FFmpeg Processing

The Lambda invokes FFmpeg from the custom Lambda Layer.

Purpose:

- Extract frame from video
- Generate thumbnail image

AWS Resource:

custom-ffmpeg-layer

Output:

- Thumbnail image generated
-

Step 11: Thumbnail Upload

The generated thumbnail is uploaded to the Thumbnail Bucket.

AWS Resource:

akshay-video-thumbnails-2026

Example:

thumbnails/sample-video.jpg

Output:

- Thumbnail stored permanently
-

Step 12: DynamoDB Update

The Lambda updates the metadata record inside DynamoDB.

AWS Resource:

videos table

Updated Fields:

- Processing status
- Thumbnail URL
- Completion information

Status Changes:

Processing → Completed

Output:

- Metadata updated successfully
-

6.5 Video Retrieval Workflow

Step 13: User Requests Video List

The frontend requests available videos.

AWS Resource:

list-videos Lambda

Purpose:

- Retrieve metadata

Output:

- List request generated
-

Step 14: DynamoDB Query

The List Videos Lambda retrieves records from DynamoDB.

Retrieved Data:

- Video ID
- Video Name
- Processing Status
- Thumbnail Information

Output:

- Metadata returned
-

Step 15: Frontend Displays Videos

The frontend displays:

- Video title
- Thumbnail image
- Processing status

User View:

- Uploaded Videos section

Output:

- Video catalog displayed
-

6.6 Video Viewing Workflow

Step 16: User Selects Video

The user clicks on a video to watch it.

Output:

- View request initiated
-

Step 17: Frontend Requests View URL

The frontend calls the Generate View URL Lambda.

AWS Resource:

generate-view-url Lambda

Purpose:

- Generate secure viewing access

Output:

- Request processed
-

Step 18: Pre-Signed View URL Generated

The Lambda creates a temporary S3 viewing URL.

Benefits:

- Secure access
- Private bucket protection
- Temporary authorization

Output:

- Viewing URL returned
-

Step 19: Video Streaming

The frontend uses the generated URL to stream the video directly from Amazon S3.

AWS Resource:

Upload Bucket

Output:

- Video playback begins
-

6.7 Failure Handling Workflow

Step 20: Processing Failure Occurs

If video processing fails:

Possible Causes:

- Corrupted file
- Timeout
- Missing resource

- Processing error

Output:

- Message remains unprocessed
-

Step 21: SQS Retry Mechanism

Amazon SQS automatically retries message processing.

Responsibilities:

- Reattempt processing
- Prevent temporary failures from causing message loss

Output:

- Retry attempts performed
-

Step 22: Message Moved to DLQ

After maximum retry attempts are exhausted, the message is moved to the Dead Letter Queue.

AWS Resource:

video-processing-dlq

Purpose:

- Preserve failed messages

Output:

- Failed message isolated
-

Step 23: CloudWatch Alarm Triggered

CloudWatch monitors the DLQ.

Alarm:

DLQ-Messages-Alarm

Trigger Condition:

DLQ Message Count ≥ 1

Output:

- Failure alert generated
-

Step 24: Failure Investigation

Developers use CloudWatch Logs and DLQ messages to identify and resolve issues.

Tools:

- CloudWatch Logs
- DLQ Messages

Output:

- Root cause identified
 - Issue resolved
-

6.8 Complete Workflow Summary

1. User uploads video.
2. Frontend requests upload URL.
3. Lambda generates pre-signed URL.
4. Video uploaded to S3.
5. S3 creates upload event.
6. Event sent to SQS.
7. Event Source Mapping invokes Lambda.
8. Thumbnail Generator downloads video.
9. FFmpeg generates thumbnail.
10. Thumbnail uploaded to S3.
11. DynamoDB metadata updated.
12. User requests video list.
13. Metadata retrieved from DynamoDB.
14. Videos displayed in frontend.
15. User selects video.
16. Generate View URL Lambda creates viewing URL.

17. Video streamed from S3.
18. Failures are handled through SQS retries, DLQ, and CloudWatch monitoring.

7. Terraform Infrastructure

7.1 Overview

Infrastructure as Code (IaC) is an approach that enables cloud resources to be provisioned, configured, and managed through code rather than manual configuration. In this project, Terraform is used as the Infrastructure as Code tool to automate the deployment and management of AWS resources.

Terraform ensures that the entire infrastructure can be recreated consistently across environments, reducing manual errors and improving maintainability. All infrastructure configurations are stored as code and version-controlled using Git and GitHub.

7.2 Purpose of Using Terraform

Terraform was selected for this project to achieve the following objectives:

- Automate AWS resource creation
- Eliminate repetitive manual configuration
- Enable version control of infrastructure
- Maintain infrastructure consistency
- Support reproducible deployments
- Simplify future modifications
- Follow Infrastructure as Code best practices

Without Terraform, each AWS service would need to be manually configured through the AWS Management Console, which increases the risk of configuration drift and human error.

7.3 Terraform Configuration Structure

The Terraform project is organized into multiple configuration files.

`provider.tf`

Purpose:

Defines the AWS provider and region configuration.

Responsibilities:

- Connect Terraform to AWS
 - Specify deployment region
 - Configure provider settings
-

variables.tf

Purpose:

Stores configurable values used throughout the infrastructure.

Examples:

- Upload bucket name
- Thumbnail bucket name
- DynamoDB table name

Benefits:

- Reusability
 - Easier maintenance
 - Reduced code duplication
-

main.tf

Purpose:

Contains the primary infrastructure definitions.

Responsibilities:

- Define AWS resources
 - Configure relationships between services
 - Manage resource dependencies
-

outputs.tf

Purpose:

Displays important infrastructure information after deployment.

Examples:

- Upload bucket name
- Thumbnail bucket name
- DynamoDB table name

Benefits:

- Easy access to resource information
- Simplified verification

7.4 AWS Resources Managed by Terraform

The following AWS resources are fully managed through Terraform.

Amazon S3 Buckets

Terraform provisions and manages:

1. Upload Bucket
akshay-video-upload-bucket-2026
2. Thumbnail Bucket
akshay-video-thumbnails-2026

Responsibilities:

- Bucket creation
- Configuration management
- State tracking

Amazon DynamoDB

Terraform provisions:

Table Name:

videos

Configuration:

- Partition Key: video_id
- Billing Mode: PAY_PER_REQUEST

Responsibilities:

- Table creation
 - Schema definition
 - Capacity configuration
-

Amazon SQS

Terraform provisions:

1. video-processing-queue
2. video-processing-dlq

Configuration:

- Visibility timeout
- Message retention period
- Queue settings

Responsibilities:

- Queue creation
 - Queue configuration
 - Dependency management
-

AWS Lambda Functions

Terraform manages deployment and configuration of:

1. generate-upload-url
2. generate-view-url
3. list-videos
4. thumbnail-creation
5. video-thumbnail-generator-v2
6. video-failure-handler

Managed Configuration:

- Runtime
- Handler
- Memory allocation
- Timeout settings
- Environment variables
- Lambda layers
- Deployment packages

Lambda Event Source Mapping

Terraform manages the connection between:

video-processing-queue

and

video-thumbnail-generator-v2

Configuration:

- Batch Size = 1
- Enabled = True

Purpose:

- Automatic queue polling
- Lambda invocation

S3 Event Notifications

Terraform manages upload bucket notifications.

Configuration:

Event Type:

s3:ObjectCreated:*

Filters:

- Prefix: videos/
- Suffix: .mp4

Destination:

video-processing-queue

Purpose:

- Trigger video processing pipeline automatically
-

CloudWatch Alarm

Terraform manages:

DLQ-Messages-Alarm

Purpose:

Monitor messages arriving in the Dead Letter Queue.

Condition:

Visible Messages $\geq$ 1

Benefits:

- Failure detection
 - Operational monitoring
-

7.5 Terraform State Management

Terraform maintains a state file that tracks deployed infrastructure resources.

Purpose:

- Maintain resource mappings
- Detect infrastructure changes
- Support incremental updates

State Files:

- terraform.tfstate
- terraform.tfstate.backup

Terraform compares the desired configuration with the actual AWS environment before applying any changes.

7.6 Terraform Workflow Used in the Project

The following workflow is used when making infrastructure changes.

Step 1

Modify Terraform configuration files.

Example:

- Add a new resource
 - Update existing resource
-

Step 2

Validate configuration.

Command:

```
terraform validate
```

Purpose:

Detect syntax and configuration errors.

Step 3

Generate execution plan.

Command:

```
terraform plan
```

Purpose:

Preview changes before deployment.

Step 4

Apply infrastructure changes.

Command:

```
terraform apply
```

Purpose:

Deploy modifications to AWS.

Step 5

Verify infrastructure state.

Command:

```
terraform plan
```

Expected Result:

No changes.

This confirms that the deployed infrastructure matches the Terraform configuration.

7.7 Benefits Achieved Through Terraform

By using Terraform, the project gains several advantages:

- Infrastructure version control
- Faster deployments
- Improved consistency
- Reduced human errors
- Easier maintenance
- Reproducible environments
- Simplified disaster recovery
- Better collaboration

Terraform transforms infrastructure management from a manual process into an automated, repeatable, and reliable workflow, making it an essential component of the project's cloud architecture.

8. Security Implementation

8.1 Overview

Security is a critical aspect of cloud-based applications, particularly when handling user-uploaded content and storing data in cloud services. The Mini Video Processing Backend incorporates multiple security mechanisms to protect resources, restrict unauthorized access, and ensure secure communication between services.

The primary security measures implemented in this project include:

- IAM Roles and Permissions
- Pre-signed URLs
- Private Amazon S3 Buckets

These mechanisms help ensure that users and services only have access to the resources they require while maintaining the confidentiality and integrity of stored data.

8.2 IAM Roles and Permissions

What is IAM?

AWS Identity and Access Management (IAM) is a service that enables secure control of access to AWS resources.

IAM allows permissions to be assigned through roles and policies rather than embedding credentials directly into applications.

IAM Roles Used in the Project

Each Lambda function operates using its own IAM execution role.

Examples:

- generate-upload-url-role
- generate-view-url-role
- list-videos-role
- thumbnail-creation-role

- video-thumbnail-generator-v2-role
- video-failure-handler-role

These roles provide temporary AWS credentials whenever a Lambda function executes.

Permissions Granted

Depending on the function's responsibilities, IAM roles are granted access to specific AWS services.

Examples:

Amazon S3 Access

Required for:

- Uploading videos
- Downloading videos
- Uploading thumbnails
- Generating pre-signed URLs

Actions:

- GetObject
 - PutObject
 - ListBucket
-

DynamoDB Access

Required for:

- Reading metadata
- Updating processing status
- Storing thumbnail information

Actions:

- GetItem
 - PutItem
 - UpdateItem
 - Scan
-

SQS Access

Required for:

- Receiving messages
- Processing queue events

Actions:

- ReceiveMessage
 - DeleteMessage
 - GetQueueAttributes
-

CloudWatch Logs Access

Required for:

- Writing execution logs
- Monitoring Lambda activity

Actions:

- CreateLogGroup
 - CreateLogStream
 - PutLogEvents
-

Benefits of IAM Roles

- No hardcoded AWS credentials
 - Temporary credentials generated automatically
 - Centralized permission management
 - Improved security posture
 - Reduced risk of credential leakage
-

8.3 Pre-Signed URLs

Purpose

The system uses Amazon S3 Pre-Signed URLs to provide secure temporary access to private objects without exposing AWS credentials.

A pre-signed URL is a URL that grants limited-time access to an S3 object.

Upload Pre-Signed URLs

Used By:

generate-upload-url Lambda

Workflow:

1. User requests upload access.
2. Lambda generates a pre-signed upload URL.
3. URL is returned to frontend.
4. Frontend uploads directly to S3.

Benefits:

- No need to expose AWS credentials.
 - Backend servers do not handle large video files.
 - Upload access expires automatically.
-

View Pre-Signed URLs

Used By:

generate-view-url Lambda

Workflow:

1. User selects a video.
2. Lambda generates a temporary viewing URL.
3. Frontend receives the URL.
4. Video is streamed securely from S3.

Benefits:

- Secure access to private videos.
 - Time-limited authorization.
 - Controlled resource access.
-

Security Advantages

Pre-signed URLs provide:

- Temporary access control
- Reduced attack surface
- Protection of AWS credentials
- Fine-grained object-level access

Even if a URL is exposed, access automatically expires after the configured time period.

8.4 Private Amazon S3 Buckets

Purpose

The project's S3 buckets are configured as private resources.

Buckets:

- akshay-video-upload-bucket-2026
- akshay-video-thumbnails-2026

Public access is not granted directly to users.

Why Private Buckets Are Important

If buckets were public:

- Anyone could download videos.
- Anyone could access thumbnails.
- Sensitive content could be exposed.
- Data could be indexed by search engines.

Private buckets prevent these risks.

Access Control Mechanism

Users cannot directly browse bucket contents.

Instead:

1. Upload access is provided through pre-signed upload URLs.
2. Viewing access is provided through pre-signed viewing URLs.
3. Lambda functions access buckets through IAM roles.

This creates a controlled access model where every operation is authenticated and authorized.

8.5 Secure Serverless Architecture

The project follows several AWS security best practices.

Principle of Least Privilege

IAM roles are granted only the permissions required for their tasks.

Benefits:

- Reduced attack surface
 - Limited impact of compromised resources
-

No Embedded Credentials

AWS credentials are never stored inside:

- Frontend code
- Lambda source code
- Git repositories

Benefits:

- Prevents credential leakage
 - Improves maintainability
-

Temporary Access

Pre-signed URLs provide temporary access rather than permanent permissions.

Benefits:

- Reduced security risk
- Automatic expiration

Managed AWS Security

The project benefits from AWS-managed security features such as:

- IAM authentication
- S3 access controls
- Lambda execution isolation
- CloudWatch monitoring

8.6 Security Summary

The Mini Video Processing Backend implements multiple layers of security to protect cloud resources and user content. IAM roles provide secure service-to-service communication, pre-signed URLs enable temporary controlled access to objects, and private S3 buckets prevent unauthorized access to stored videos and thumbnails.

These mechanisms collectively ensure that the application follows modern cloud security best practices while maintaining usability, scalability, and operational efficiency.

9. Monitoring and Observability

9.1 Overview

Monitoring is essential for ensuring the reliability, performance, and availability of cloud applications. In this project, AWS CloudWatch is used to monitor system activity, track application behavior, detect failures, and assist with troubleshooting.

The monitoring architecture consists of:

- CloudWatch Logs
- CloudWatch Metrics
- Dead Letter Queue (DLQ) Monitoring
- CloudWatch Alarm

These services provide visibility into the complete video processing pipeline.

9.2 CloudWatch Logs

Purpose

AWS CloudWatch Logs collects and stores logs generated by Lambda functions during execution.

Every Lambda function in the project automatically writes logs to CloudWatch.

This enables developers to:

- Monitor application behavior
- Debug issues
- Track execution flow
- Analyze failures
- Verify successful processing

Lambda Functions Monitored

The following Lambda functions generate CloudWatch logs:

1. generate-upload-url
2. generate-view-url
3. list-videos
4. thumbnail-creation
5. video-thumbnail-generator-v2
6. video-failure-handler

Information Captured in Logs

CloudWatch logs record various execution details, including:

Function Invocation

Shows when a Lambda function is executed.

Example:

- Video upload request received
- Thumbnail generation started
- Metadata update initiated

Processing Status

Tracks the progress of video processing.

Examples:

- Video downloaded successfully
- Thumbnail created successfully
- Thumbnail uploaded successfully
- DynamoDB updated successfully

Error Messages

Captures runtime exceptions and failures.

Examples:

- Missing object in S3
- Invalid event payload
- Access denied errors
- Timeout exceptions

Performance Metrics

CloudWatch automatically records:

- Invocation count
- Execution duration
- Memory usage
- Error count

These metrics help evaluate system performance and identify bottlenecks.

9.3 CloudWatch Log Groups

Each Lambda function has its own dedicated log group.

Examples:

- /aws/lambda/generate-upload-url
- /aws/lambda/generate-view-url
- /aws/lambda/list-videos
- /aws/lambda/thumbnail-creation
- /aws/lambda/video-thumbnail-generator-v2
- /aws/lambda/video-failure-handler

This separation simplifies troubleshooting and log analysis.

9.4 Dead Letter Queue (DLQ)

Purpose

The project uses a Dead Letter Queue (DLQ) to capture failed video-processing messages.

Queue Name:

video-processing-dlq

The DLQ acts as a safety mechanism that prevents failed messages from being lost.

Failure Scenario

A message is sent to the DLQ when:

1. Video upload event reaches SQS.
2. SQS triggers the thumbnail generation Lambda.
3. Processing repeatedly fails.
4. Maximum retry attempts are exceeded.
5. Message is moved to the DLQ.

This ensures that failed processing events can be investigated later.

Benefits of DLQ

- Prevents message loss
- Preserves failed events
- Supports troubleshooting
- Improves system reliability
- Enables operational monitoring

9.5 DLQ CloudWatch Alarm

Purpose

A CloudWatch alarm continuously monitors the Dead Letter Queue.

Alarm Name:

DLQ-Messages-Alarm

The alarm detects whether failed messages are accumulating inside the DLQ.

Metric Monitored

CloudWatch Metric:

ApproximateNumberOfMessagesVisible

Queue:

video-processing-dlq

This metric represents the number of messages waiting in the Dead Letter Queue.

Alarm Condition

The alarm is triggered when:

Visible Messages $\geq$ 1

This indicates that at least one message has failed processing and requires attention.

Alarm States

OK State

Condition:

No messages exist in the DLQ.

Meaning:

- Video processing is operating normally.
 - No failed events require investigation.
-

ALARM State

Condition:

One or more messages exist in the DLQ.

Meaning:

- Processing failure occurred.
 - Investigation is required.
-

9.6 Monitoring Workflow

The monitoring process operates as follows:

1. User uploads a video.
2. S3 generates an event.
3. Event is sent to SQS.
4. SQS triggers thumbnail generation.
5. Lambda processes the video.
6. CloudWatch records execution logs.
7. Failed messages are moved to the DLQ.
8. CloudWatch Alarm monitors the DLQ.
9. Alarm enters ALARM state if failures occur.

This provides end-to-end visibility into the entire processing pipeline.

9.7 Benefits of Monitoring Implementation

The monitoring architecture provides several advantages:

- Real-time visibility into system activity
- Faster issue detection
- Simplified troubleshooting
- Failure tracking through DLQ

- Improved operational reliability
- Better system observability
- Enhanced maintenance and support

By combining CloudWatch Logs with DLQ monitoring and CloudWatch Alarms, the project achieves a robust monitoring framework that ensures reliable operation of the serverless video-processing pipeline.

10. Testing and Validation

10.1 Overview

Testing was performed throughout the development process to verify that all components of the Mini Video Processing Backend functioned correctly. The objective was to validate the complete video-processing workflow, ensure accurate thumbnail generation, verify metadata updates, and confirm that failure-handling mechanisms operated as expected.

The testing process focused on three primary areas:

- Video Upload Testing
- Thumbnail Generation Testing
- Failure Handling Testing

10.2 Upload Testing

Objective

To verify that users can successfully upload video files and that the uploaded files are stored correctly in Amazon S3.

Test Procedure

1. Open the frontend application.
 2. Select an MP4 video file.
 3. Click the upload button.
 4. Frontend requests a pre-signed upload URL.
 5. Upload video using the generated URL.
 6. Verify file existence in the upload bucket.
-

Expected Result

- Upload URL generated successfully.
 - Video uploaded without errors.
 - Object stored inside the upload bucket.
 - S3 Object Created event generated.
-

AWS Resources Tested

- React Frontend
 - generate-upload-url Lambda
 - Amazon S3 Upload Bucket
-

Test Result

✓ Passed

The uploaded video was successfully stored in Amazon S3 and became available for further processing.

10.3 Thumbnail Generation Testing

Objective

To verify that uploaded videos automatically trigger thumbnail generation and metadata updates.

Test Procedure

1. Upload a video file.
 2. Verify S3 upload completion.
 3. Confirm S3 event notification creation.
 4. Confirm event delivery to SQS queue.
 5. Verify Lambda invocation.
 6. Confirm thumbnail generation.
 7. Verify thumbnail upload.
 8. Verify DynamoDB update.
-

Expected Result

- Video processing starts automatically.
 - Thumbnail generated successfully.
 - Thumbnail uploaded to thumbnail bucket.
 - DynamoDB record updated.
 - Processing status changes from Processing to Completed.
-

AWS Resources Tested

- S3 Event Notification
 - Amazon SQS
 - Event Source Mapping
 - video-thumbnail-generator-v2 Lambda
 - FFmpeg Layer
 - Thumbnail Bucket
 - DynamoDB
-

Validation Steps

S3 Upload Bucket

Verified uploaded video exists.

Result:

✓ Passed

SQS Queue

Verified upload events are delivered successfully.

Result:

✓ Passed

Thumbnail Generator Lambda

Verified successful invocation through CloudWatch Logs.

Result:

✓ Passed

Thumbnail Bucket

Verified thumbnail image creation.

Result:

✓ Passed

DynamoDB

Verified metadata update.

Fields Confirmed:

- video_id
- status
- thumbnail_url

Result:

✓ Passed

Test Result

✓ Passed

The complete automated video-processing pipeline executed successfully and produced thumbnails for uploaded videos.

10.4 Video Listing Testing

Objective

To verify that uploaded videos are correctly retrieved and displayed in the frontend.

Test Procedure

1. Upload multiple videos.
2. Allow processing to complete.
3. Open the application.
4. Request video list.

Expected Result

- Videos retrieved from DynamoDB.
- Metadata displayed correctly.
- Processing status shown.
- Thumbnails displayed.

AWS Resources Tested

- list-videos Lambda
- DynamoDB
- Frontend Application

Test Result

✓ Passed

Video records were successfully retrieved and displayed in the user interface.

10.5 Video Playback Testing

Objective

To verify secure video playback using pre-signed URLs.

Test Procedure

1. Select an uploaded video.

2. Request viewing URL.
 3. Open generated URL.
 4. Stream video from S3.
-

Expected Result

- View URL generated successfully.
 - Video streamed correctly.
 - No direct public access required.
-

AWS Resources Tested

- generate-view-url Lambda
 - S3 Upload Bucket
-

Test Result

✓ Passed

Videos were streamed successfully using temporary pre-signed URLs.

10.6 Failure Handling Testing

Objective

To verify that failed processing events are handled correctly and can be monitored.

Test Procedure

1. Monitor processing pipeline.
 2. Observe Lambda execution logs.
 3. Verify SQS retry behavior.
 4. Verify DLQ configuration.
 5. Verify CloudWatch alarm monitoring.
-

Expected Result

- Failed messages retained.
 - Retries performed automatically.
 - Failed events moved to DLQ.
 - CloudWatch monitoring available.
-

AWS Resources Tested

- Amazon SQS
 - Dead Letter Queue
 - CloudWatch Logs
 - CloudWatch Alarm
-

Validation Steps

CloudWatch Logs

Verified that execution logs are generated for Lambda functions.

Result:

✓ Passed

Dead Letter Queue

Verified DLQ configuration and message preservation capability.

Result:

✓ Passed

CloudWatch Alarm

Verified alarm configuration monitoring DLQ message count.

Alarm:

DLQ-Messages-Alarm

Result:

✓ Passed

10.7 Testing Summary

Test Case	Status
Video Upload	Passed
S3 Event Notification	Passed
SQS Message Delivery	Passed
Lambda Invocation	Passed
Thumbnail Generation	Passed
Thumbnail Upload	Passed
DynamoDB Update	Passed
Video Listing	Passed
Video Playback	Passed
CloudWatch Logging	Passed
DLQ Monitoring	Passed
CloudWatch Alarm	Passed

10.8 Conclusion

All major functionalities of the Mini Video Processing Backend were successfully tested and validated. The system demonstrated reliable operation across video uploads, automated thumbnail generation, metadata management, video playback, monitoring, and failure handling. The testing results confirm that the architecture functions as intended and meets the project's design objectives.

11. Challenges Faced and Solutions

11.1 Overview

During the development of the Mini Video Processing Backend, several technical challenges were encountered while implementing the serverless video-processing pipeline. These challenges involved AWS Lambda configuration, event-driven architecture, Terraform infrastructure management, and source code version control.

Each issue provided valuable learning opportunities and contributed to a deeper understanding of AWS cloud services and Infrastructure as Code practices.

11.2 Lambda Timeout Issues

Challenge

During thumbnail generation, Lambda executions occasionally timed out before completing video processing.

The thumbnail generation process involved:

- Downloading video from S3
- Running FFmpeg
- Generating thumbnail
- Uploading thumbnail
- Updating DynamoDB

These operations required more execution time than initially configured.

Symptoms

- Lambda execution terminated unexpectedly
 - Thumbnail not generated
 - Processing status remained incomplete
 - CloudWatch logs showed timeout errors
-

Root Cause

The default Lambda timeout configuration was insufficient for video-processing workloads.

Large video files required additional processing time.

Solution

The Lambda timeout value was increased to provide sufficient execution time for:

- Video download
- FFmpeg processing
- Thumbnail upload
- Database update

CloudWatch Logs were used to monitor execution duration and validate the new configuration.

Outcome

✓ Thumbnail generation completed successfully without timeout failures.

11.3 Event Source Mapping Issues

Challenge

Uploaded videos remained in the "Processing" state even though S3 events and SQS messages were being generated correctly.

The thumbnail generation Lambda was not being triggered automatically.

Symptoms

- Videos uploaded successfully
 - Messages visible in SQS
 - No Lambda execution logs generated
 - No thumbnails created
-

Root Cause

The Lambda Event Source Mapping between:

- video-processing-queue
- video-thumbnail-generator-v2

was not fully managed by Terraform initially.

The queue existed and messages were arriving, but Lambda was not consuming them properly.

Solution

The Event Source Mapping was imported into Terraform and added to the Terraform configuration.

Verification steps included:

- Terraform import
 - Terraform plan
 - Terraform apply
 - CloudWatch validation
-

Outcome

✓ SQS messages were automatically consumed and video processing started successfully.

11.4 S3 Event Notification Management

Challenge

Videos uploaded to the S3 bucket were not always triggering the complete processing workflow through Terraform-managed infrastructure.

Symptoms

- Upload successful
 - Processing pipeline not initiated
 - No new messages arriving in SQS
-

Root Cause

The S3 Event Notification configuration existed in AWS but was not initially tracked by Terraform.

This created a mismatch between the actual infrastructure and Terraform state.

Solution

The S3 Bucket Notification resource was imported and managed through Terraform.

Notification Configuration:

- Event: s3:ObjectCreated:*
 - Prefix: videos/
 - Suffix: .mp4
 - Destination: video-processing-queue
-

Outcome

✓ Every uploaded MP4 video automatically entered the processing pipeline.

11.5 Terraform Import Challenges

Challenge

Several AWS resources were originally created manually through the AWS Console before Terraform management was introduced.

These resources included:

- Lambda Event Source Mapping
 - CloudWatch Alarm
 - S3 Event Notification
-

Symptoms

Terraform attempted to create resources that already existed.

Example Errors:

- Resource already exists
 - Duplicate configuration warnings
-

Root Cause

Terraform state was unaware of manually created AWS resources.

Solution

Terraform import commands were used to associate existing AWS resources with Terraform state.

Process:

1. Identify resource ID.
 2. Import resource.
 3. Run Terraform plan.
 4. Resolve configuration differences.
 5. Apply changes.
-

Outcome

✓ Existing infrastructure became fully synchronized with Terraform state.

11.6 Git Large File Issues

Challenge

While pushing the project to GitHub, the push operation failed due to extremely large files.

Symptoms

GitHub displayed errors such as:

- File exceeds GitHub size limit

- Push rejected
 - Large files detected
-

Root Cause

Terraform provider binaries inside:

terraform/.terraform/

were accidentally committed to Git.

These provider files were hundreds of megabytes in size and exceeded GitHub's 100 MB file limit.

Solution

The following actions were performed:

1. Created a root-level .gitignore file.
2. Added Terraform-generated files to .gitignore.
3. Removed provider binaries from Git tracking.
4. Removed Terraform state files from Git tracking.
5. Recommitted clean project files.

Ignored Files:

- .terraform/
 - *.tfstate
 - *.tfstate.backup
 - *.zip
 - node_modules/
-

Outcome

✓ Repository size was reduced significantly and GitHub push operations completed successfully.

11.7 Lambda Deployment Package Issues

Challenge

After deploying the thumbnail generator Lambda, execution failed immediately.

Symptoms

CloudWatch Logs displayed:

Runtime.ImportModuleError

Error: Cannot find module 'index'

Root Cause

The Lambda handler configuration did not match the actual deployment package structure.

AWS Lambda could not locate the correct entry file.

Solution

The handler configuration was updated to correctly reference:

lambda_function.handler

The deployment package was recreated and redeployed.

Outcome

✓ Lambda initialization succeeded and thumbnail generation resumed normally.

11.8 Infrastructure Drift Management

Challenge

Several AWS resources were modified manually during development and testing.

This created differences between:

- Actual AWS Infrastructure
 - Terraform Configuration
-

Symptoms

Terraform plan reported unexpected changes.

Root Cause

Manual console changes were not reflected in Terraform configuration.

Solution

Terraform became the single source of truth for infrastructure management.

Validation process:

- terraform plan
- terraform apply
- terraform plan (verification)

Expected Result:

No changes.

Outcome

✓ Infrastructure became fully synchronized and reproducible.

11.9 Key Learnings

The challenges encountered during development provided practical experience with:

- AWS Lambda troubleshooting

- Event-driven architectures
- SQS integration
- S3 event notifications
- CloudWatch monitoring
- Terraform state management
- Infrastructure as Code principles
- Git repository management

These experiences significantly improved understanding of building and maintaining production-style serverless applications on AWS.

11.10 Conclusion

Although several technical challenges were encountered throughout development, each issue was successfully identified, analyzed, and resolved. The solutions implemented improved the reliability, maintainability, and scalability of the system while providing valuable hands-on experience with AWS services and Terraform-based infrastructure management.

12. Future Enhancements

12.1 Overview

The current implementation of the Mini Video Processing Backend successfully demonstrates a complete serverless video-processing workflow using AWS services and Terraform. However, several enhancements can be incorporated in future versions to improve automation, security, scalability, maintainability, and production readiness.

The following enhancements have been identified for future development.

12.2 CI/CD Pipeline Integration

Current State

Application deployment and infrastructure updates are currently performed manually using Git and Terraform commands.

Examples:

- Git push
- Terraform plan
- Terraform apply
- Manual Lambda package updates

Proposed Enhancement

Implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions.

Possible Workflow:

1. Developer pushes code to GitHub.
2. GitHub Actions automatically runs validation checks.
3. Terraform plan is executed.
4. Infrastructure changes are deployed.
5. Lambda functions are packaged automatically.
6. Updated code is deployed to AWS.

Benefits

- Faster deployments
- Reduced manual effort
- Consistent deployment process
- Early detection of errors
- Improved reliability

12.3 Remote Terraform State Management

Current State

Terraform state is currently stored locally on the developer machine.

Files:

- terraform.tfstate
- terraform.tfstate.backup

Proposed Enhancement

Store Terraform state remotely in an Amazon S3 bucket and use DynamoDB for state locking.

Architecture:

Terraform → S3 Backend → DynamoDB Lock Table

Benefits

- Centralized state management
 - Team collaboration support
 - State backup and recovery
 - Prevents concurrent modifications
 - Improved infrastructure reliability
-

12.4 Terraform-Managed IAM Roles and Policies

Current State

Some IAM roles and policies were created manually through the AWS Management Console.

Although the infrastructure is largely managed by Terraform, IAM resources are not yet fully controlled through Infrastructure as Code.

Proposed Enhancement

Manage all IAM resources through Terraform.

Resources to Manage:

- Lambda execution roles
 - IAM policies
 - Policy attachments
 - Role permissions
-

Benefits

- Complete Infrastructure as Code implementation
 - Improved reproducibility
 - Easier auditing
 - Simplified permission management
 - Reduced configuration drift
-

12.5 User Authentication and Authorization

Current State

The application currently allows video uploads and access without user authentication.

The focus of the current project is backend processing and infrastructure automation.

Proposed Enhancement

Implement user authentication using AWS Cognito.

Possible Features:

- User registration
 - User login
 - Password management
 - Token-based authentication
 - Session management
-

Authorization Features

Future versions can support:

- User-specific video libraries
 - Role-based access control
 - Private user content
 - Ownership validation
-

Benefits

- Improved security
 - Personalized user experience
 - Controlled access to resources
 - Multi-user support
-

12.6 Advanced Video Processing Features

Proposed Enhancements

Additional media-processing capabilities can be introduced.

Examples:

- Multiple thumbnail generation
 - Video compression
 - Video transcoding
 - Resolution conversion
 - Watermarking
 - Preview clip generation
-

Benefits

- Enhanced functionality
 - Improved user experience
 - Production-ready media workflows
-

12.7 Enhanced Monitoring and Alerting

Current State

Monitoring currently includes:

- CloudWatch Logs
 - DLQ Monitoring
 - CloudWatch Alarm
-

Proposed Enhancements

- SNS notifications for failures
 - Email alerts
 - Slack notifications
 - Dashboard creation
 - Custom CloudWatch metrics
-

Benefits

- Faster incident response
 - Better operational visibility
 - Improved observability
-

12.8 API Management

Proposed Enhancement

Introduce Amazon API Gateway as the primary entry point for backend APIs.

Features:

- Request validation
 - Rate limiting
 - API authentication
 - Usage monitoring
 - API versioning
-

Benefits

- Improved security
 - Better API governance
 - Enhanced scalability
-

12.9 Infrastructure Scalability Improvements

Proposed Enhancements

To support larger workloads, the architecture can be extended with:

- Multiple processing queues
 - Parallel Lambda processing
 - EventBridge integration
 - Auto-scaling strategies
 - Multi-region deployment
-

Benefits

- Increased throughput
 - Higher availability
 - Better fault tolerance
-

12.10 Future Vision

The current project establishes a strong foundation for a cloud-native serverless media-processing platform. By incorporating CI/CD automation, remote Terraform state management, fully Terraform-managed IAM resources, user authentication, and advanced monitoring capabilities, the system can evolve into a production-grade solution capable of supporting real-world workloads at scale.

These enhancements will further improve security, automation, maintainability, scalability, and operational efficiency while adhering to modern cloud engineering best practices.

13. Conclusion

13.1 Project Summary

The Mini Video Processing Backend project successfully demonstrates the design and implementation of a fully serverless, event-driven video-processing system on Amazon Web Services (AWS). The system automates the complete workflow from video upload to thumbnail generation and metadata management while leveraging modern cloud-native architecture principles.

The project was developed using AWS managed services, enabling scalable, reliable, and cost-effective processing without the need to manage traditional servers.

13.2 Objectives Achieved

The primary objectives of the project were successfully accomplished:

- Secure video upload using pre-signed URLs
- Storage of uploaded videos in Amazon S3
- Automated event-driven processing pipeline
- Thumbnail generation using AWS Lambda and FFmpeg
- Metadata storage and management using DynamoDB
- Video listing and retrieval functionality
- Secure video access using temporary pre-signed URLs

- Monitoring and logging through CloudWatch
- Failure handling using SQS Dead Letter Queue
- Infrastructure management using Terraform

The final solution provides an end-to-end automated workflow with minimal manual intervention.

13.3 AWS Services Utilized

The project integrates multiple AWS services working together as a complete cloud ecosystem.

Services Used:

- Amazon S3
- AWS Lambda
- Amazon DynamoDB
- Amazon SQS
- Amazon CloudWatch
- IAM
- Terraform

Each service plays a specific role within the architecture, contributing to scalability, automation, reliability, and security.

13.4 Key Technical Achievements

Several important cloud engineering concepts were implemented throughout the project:

Serverless Computing

The application uses AWS Lambda to execute processing logic without managing servers.

Event-Driven Architecture

The workflow is triggered automatically through S3 events and SQS messaging.

Infrastructure as Code

Terraform was used to provision and manage infrastructure resources consistently and reproducibly.

Secure Resource Access

Pre-signed URLs and private S3 buckets were implemented to ensure secure access to video content.

Fault-Tolerant Design

Dead Letter Queues and CloudWatch monitoring were incorporated to improve reliability and observability.

13.5 Learning Outcomes

This project provided valuable hands-on experience with modern cloud technologies and DevOps practices.

Key learnings include:

- AWS serverless architecture design
- Lambda function development and deployment
- S3 event notifications
- SQS message processing
- DynamoDB data modeling
- CloudWatch monitoring and troubleshooting
- IAM security management
- Terraform Infrastructure as Code
- Git and GitHub version control
- Debugging distributed cloud systems

The project also provided practical exposure to real-world challenges such as infrastructure drift, deployment issues, Lambda timeouts, event-source configuration, and cloud resource management.

13.6 Overall Impact

The completed solution demonstrates how multiple AWS services can be integrated to build an automated media-processing platform. The architecture is scalable, secure, maintainable, and aligned with industry best practices for cloud-native application development.

The implementation showcases the advantages of serverless computing, event-driven workflows, Infrastructure as Code, and cloud automation while providing a strong foundation for future enhancements such as CI/CD pipelines, authentication, remote Terraform state management, and advanced media-processing features.

13.7 Final Conclusion

The Mini Video Processing Backend successfully fulfills its intended purpose of automating video processing in a serverless AWS environment. Through the integration of AWS services and Terraform-based infrastructure management, the project demonstrates practical cloud engineering skills and provides a production-inspired solution that is scalable, secure, and extensible.

The knowledge and experience gained from this project contribute significantly to understanding modern cloud architectures, DevOps methodologies, and serverless application development, making it a valuable learning and portfolio project for future cloud and software engineering endeavors.